

Light up LED

1. Preface

This lesson will use software to write a program for the LED on the STM32 main control board, controlling the onboard LED to blink.

2. Getting Ready

2.1 Software Preparation

1. Install and configure Keil5.

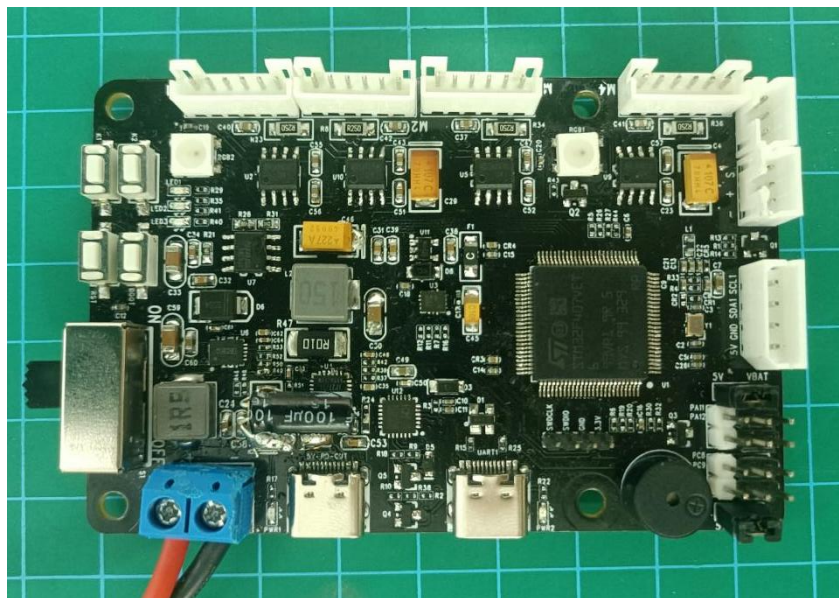
If it is not installed or configured, please refer to “2.STM32 Development Fundamentals/1.Set Development Environment”.

2. Install Jlink firmware.

If it is not installed or configured, please access “2.STM32 Development Fundamentals/1.Set Development Environment” for flashing.

2.2 Hardware Preparation

1. STM32 Main Controller

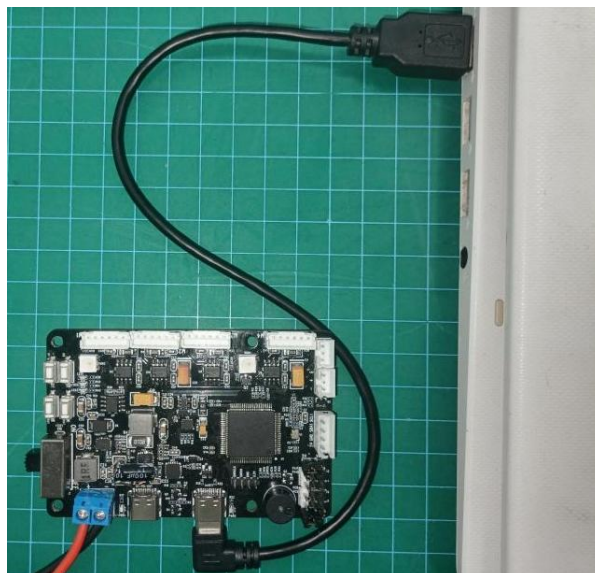


2. The Jlink debugger and a Micro-USB data cable (self-provided) are used for code flashing.



3. Operation Steps

- 1) Connect the RRCLite controller to the PC with the USB cable.



- 2) Download the program. For specific download method (Type-C serial port download), please refer to “2.STM32 Development Fundamentals/3.Project Compilation and Download”.

4. Core Program Analysis

This code is an embedded program based on the STM32 microcontroller. It controls the switching state of the LED light. The program mainly includes some header files, such as main.h, and some library files, such as GPIO library and RCC library.

In the main function, a series of functions are called to initialize the various modules of the STM32 microcontroller, including the system clock, GPIO pins, and LED lights.

```
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */
    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    /* USER CODE BEGIN 2 */
    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1) {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_10, GPIO_PIN_RESET);
        HAL_Delay(2000);

        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_10, GPIO_PIN_SET);
        HAL_Delay(2000);
    }
    /* USER CODE END 3 */
}
```

In the while loop, the “HAL_GPIO_WritePin” and “HAL_Delay” functions are called to control the LED light to blink every 2 seconds.

In the main function, the program calls the “HAL_Init” function to initialize the MCU. Then, it calls the “SystemClock_Config” and “MX_GPIO_Init” functions to configure the system clock and GPIO pins, respectively. These functions are

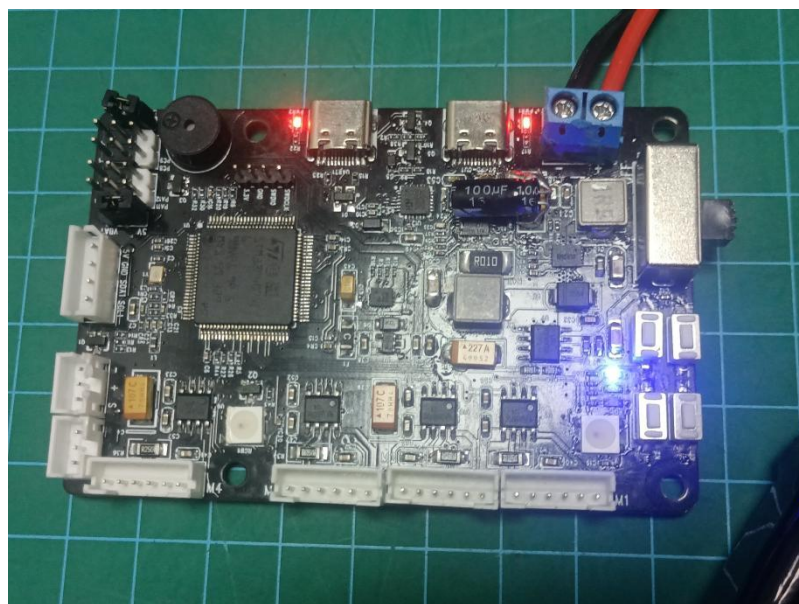
codes generated by STM32 CubeMX. They are used to initialize the various functional modules of the STM32 microcontroller.

In the while loop, the “HAL_GPIO_WritePin” and “HAL_Delay” functions are called to make the LED light blink every 2 seconds. The “HAL_GPIO_WritePin” function controls the voltage level state of the GPIO pin, and the “GPIO_PIN_RESET” and “GPIO_PIN_SET” parameters are passed in to turn off and turn on the LED light, respectively. The “HAL_Delay” function is used to implement delay, allowing the program to pause for a period of time before executing the next statement. This controls the time interval of LED light blinking.

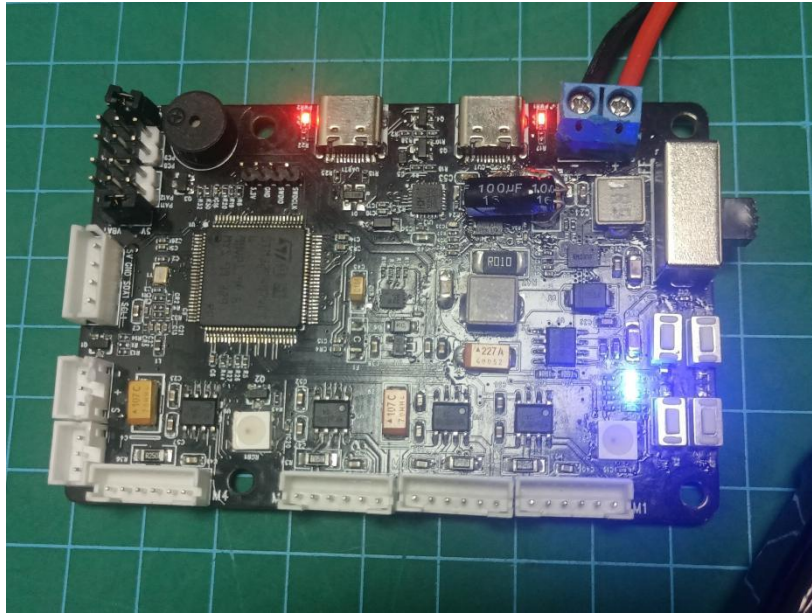
5. Program Outcome

After the program is flashed, remove the USB cable. Then, you can connect a 7.4V or 11.4V battery to the STM32.

Please ensure to connect the battery correctly, with the black wire to the negative terminal “-” and the red wire to the positive terminal “+”, as shown in the two diagrams below:



Effect Diagram 1 (LED Light Off)



Effect Diagram 2 (LED Light On)